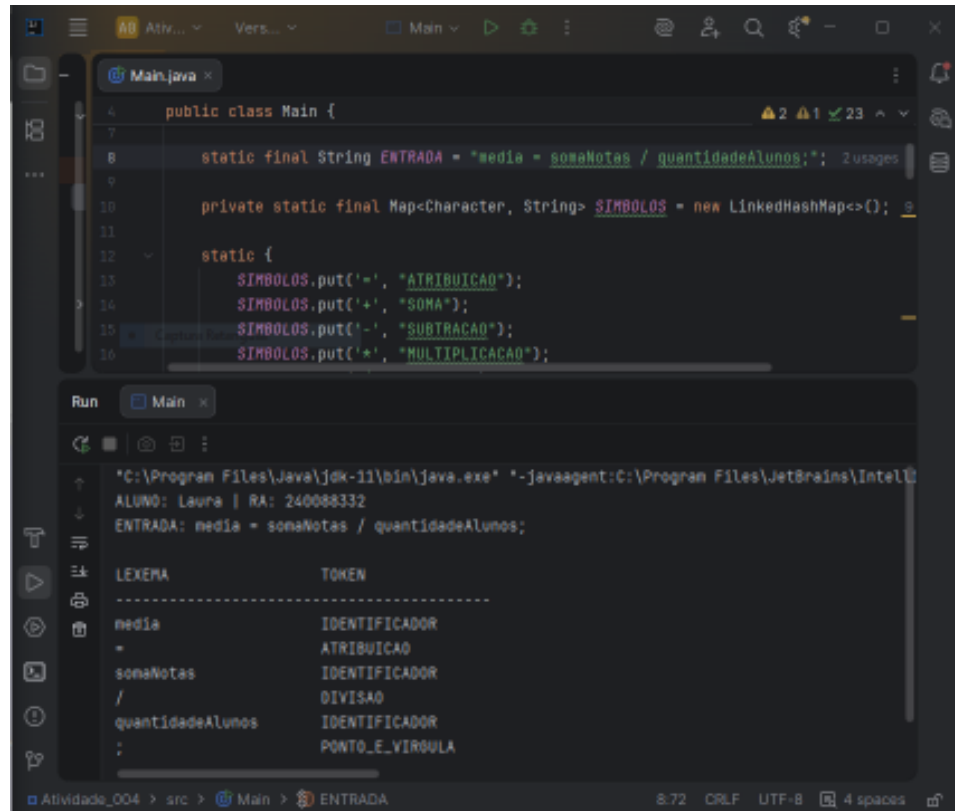


Atividade 004

Estação 01 - De caracteres a lexemas e tokens



The screenshot shows an IDE with a Java file named `Main.java`. The code defines a `Main` class with a static `ENTRADA` string and a static `SIMBOLOS` map. The map contains entries for operators: `=` (ATRIBUICAO), `+` (SOMA), `-` (SUBTRACAO), and `*` (MULTIPLICACAO).

```
public class Main {  
    static final String ENTRADA = "media = somaNotas / quantidadeAlunos;";  
    private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();  
    static {  
        SIMBOLOS.put('=', "ATRIBUICAO");  
        SIMBOLOS.put('+', "SOMA");  
        SIMBOLOS.put('-', "SUBTRACAO");  
        SIMBOLOS.put('*', "MULTIPLICACAO");  
    }  
}
```

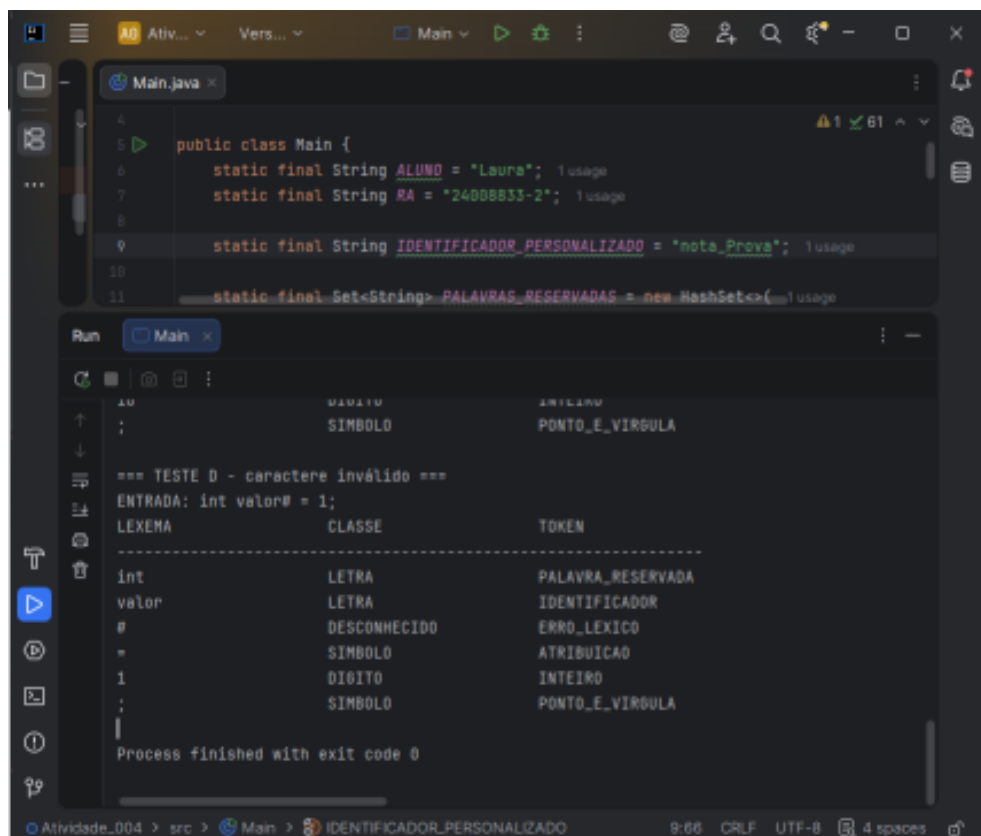
The Run console shows the execution of the program. The output is:

```
ALUNO: Laura | RA: 240088332  
ENTRADA: media = somaNotas / quantidadeAlunos;
```

Below the output, a table shows the lexemes and tokens for the input string:

LEXEMA	TOKEN
media	IDENTIFICADOR
=	ATRIBUICAO
somaNotas	IDENTIFICADOR
/	DIVISAO
quantidadeAlunos	IDENTIFICADOR
;	PONTO_E_VIRGULA

Estação 02 - O que o analisador léxico reconhece



The screenshot shows an IDE with a Java file named `Main.java`. The code defines a `Main` class with static `ALUNO`, `RA`, and `IDENTIFICADOR_PERSONALIZADO` strings, and a static `PALAVRAS_RESERVADAS` set.

```
public class Main {  
    static final String ALUNO = "Laura";  
    static final String RA = "24008833-2";  
    static final String IDENTIFICADOR_PERSONALIZADO = "nota_Prova";  
    static final Set<String> PALAVRAS_RESERVADAS = new HashSet<>();  
}
```

The Run console shows the execution of the program. The output is:

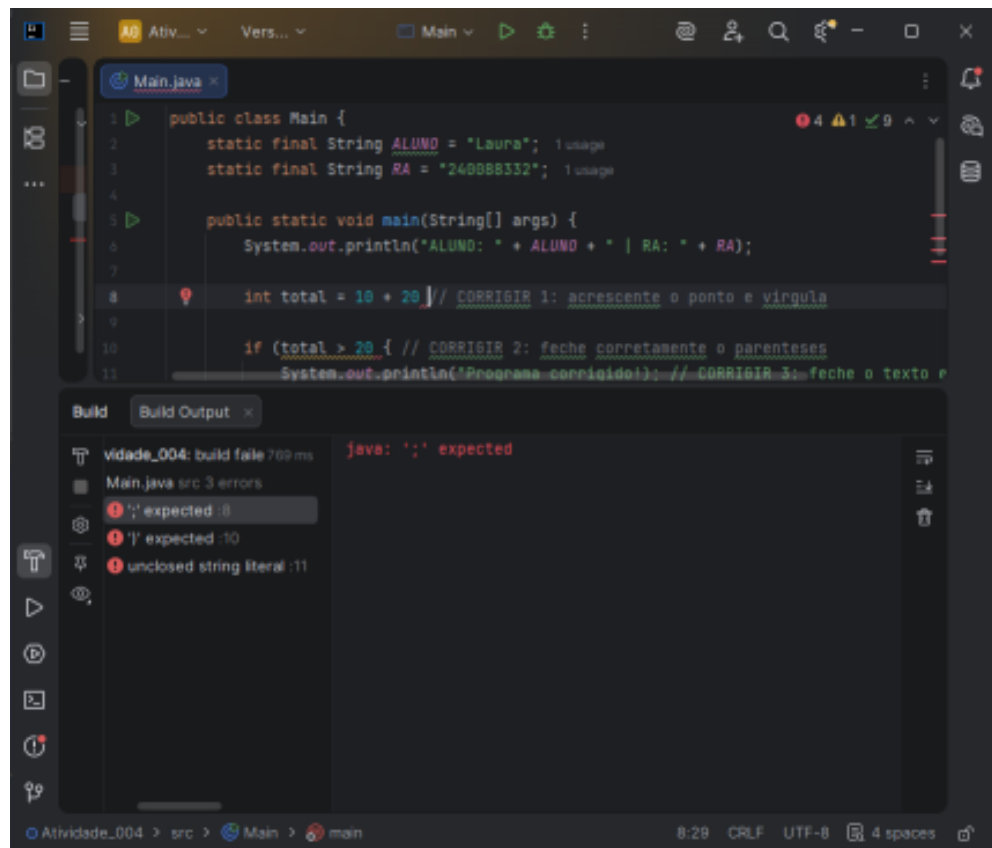
```
ALUNO: Laura | RA: 24008833-2 | IDENTIFICADOR_PERSONALIZADO: nota_Prova
```

Below the output, a table shows the lexemes and tokens for the input string:

LEXEMA	CLASSE	TOKEN
int	LETRA	PALAVRA_RESERVADA
valor	LETRA	IDENTIFICADOR
#	DESCONHECIDO	ERRO_LEXICO
=	SIMBOLO	ATRIBUICAO
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

Estação 03 - Quando o compilador encontra erros

a) Com erro:



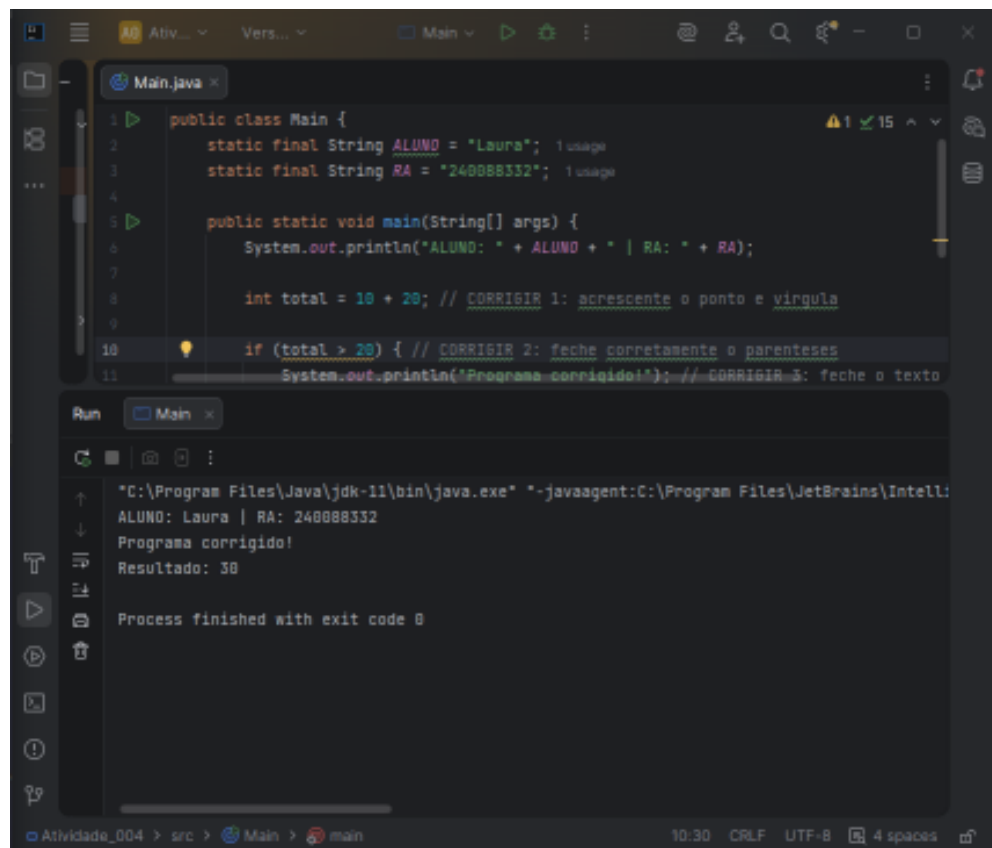
The screenshot shows an IDE with a Java file named `Main.java`. The code contains several errors highlighted by the compiler:

```
1 public class Main {
2     static final String ALUNO = "Laura"; 1 usage
3     static final String RA = "240088332"; 1 usage
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto e
```

The Build Output window shows the following errors:

- java: ';' expected
- java: ')' expected
- unclosed string literal

b) Erro corrigido:



The screenshot shows the same IDE with the corrected Java code. The errors have been resolved, and the code is now compiling and running successfully.

```
1 public class Main {
2     static final String ALUNO = "Laura"; 1 usage
3     static final String RA = "240088332"; 1 usage
4
5     public static void main(String[] args) {
6         System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
7
8         int total = 10 + 20; // CORRIGIR 1: acrescente o ponto e virgula
9
10        if (total > 20) { // CORRIGIR 2: feche corretamente o parenteses
11            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto e
```

The Run window shows the output of the program:

```
"C:\Program Files\Java\jdk-11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ..."
ALUNO: Laura | RA: 240088332
Programa corrigido!
Resultado: 30
Process finished with exit code 0
```

Estação 04 - Parser descendente recursivo

```
public class Main {
    static final String ALUNO = "Laura";
    static final String RA = "240088332";

    static final String ENTRADA = "(5 + 3) * 4";
    static final boolean DEMONSTRAR_RECURSÃO_ESQUERDA = false;

    public static void main(String[] args) {
        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
        System.out.println("ENTRADA: " + ENTRADA);
        System.out.println();
    }
}
```

Run Main

```
*C:\Program Files\Java\jdk-11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA\bin\idea-agent.jar" -Didea.config.path=C:\Program Files\JetBrains\IntelliJ IDEA\config\idea.config.xml -Didea.copyright.path=C:\Program Files\JetBrains\IntelliJ IDEA\config\idea.copyright.xml -Didea.home.path=C:\Program Files\JetBrains\IntelliJ IDEA\bin -Didea.platform.prefix=idea -jar C:\Program Files\JetBrains\IntelliJ IDEA\bin\idea.jar
ALUNO: Laura | RA: 240088332
ENTRADA: (5 + 3) * 4

ENTRANDO em <expr> | lookahead = (
  ENTRANDO em <term> | lookahead = (
    ENTRANDO em <factor> | lookahead = (
      Abrindo parenteses
      Novo lookahead: 5
    ENTRANDO em <expr> | lookahead = 5
      ENTRANDO em <term> | lookahead = 5
        ENTRANDO em <factor> | lookahead = 5
          Numero reconhecido: 5
        )
      )
    )
  )
)
```

Estação 05 - Parser ascendente: deslocar e reduzir

```
static final String ALUNO = "Laura";
static final String RA = "240088332";

public static void main(String[] args) {
    System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
    System.out.println("ENTRADA: id+id*id");
}
```

Run Main

```
ALUNO: Laura | RA: 240088332
ENTRADA: id+id*id
```

PILHA	ENTRADA	ACAO
\$ 0	id+id*id\$	SHIFT id -> estado 5
\$ 0 id 5	+id*id\$	REDUCE F -> id
\$ 0 F 3	+id*id\$	REDUCE T -> F
\$ 0 T 2	+id*id\$	REDUCE E -> T
\$ 0 E 1	+id*id\$	SHIFT + -> estado 6
\$ 0 E 1 + 6	id*id\$	SHIFT id -> estado 5
\$ 0 E 1 + 6 id 5	*id\$	REDUCE F -> id
\$ 0 E 1 + 6 F 3	*id\$	REDUCE T -> F
\$ 0 E 1 + 6 T 9	*id\$	SHIFT * -> estado 7
\$ 0 E 1 + 6 T 9 * 7	id\$	SHIFT id -> estado 5
\$ 0 E 1 + 6 T 9 * 7 id 5	\$	REDUCE F -> id
\$ 0 E 1 + 6 T 9 * 7 F 10	\$	REDUCE T -> T * F
\$ 0 E 1 + 6 T 9	\$	REDUCE E -> E + T
\$ 0 E 1	\$	ACCEPT

Process finished with exit code 0